

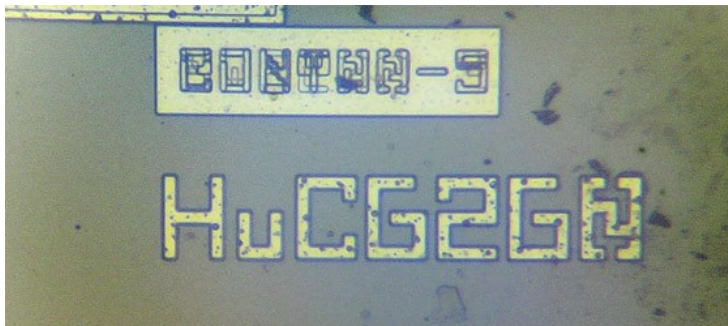


;) Home	Catégories Créations Reverse-engineering Divers Archives	Autres FE Shop Github Liens	Plus
Perso Qui ? Contact	Recherche 		

Cette page a au moins 5 ans !
This page is at least 5 years old !

Dumping the NEC PC Engine YUV table

This page is about a small part of a long story which involved several people.
Its chronology is laid out in detail in [this RetroRGB post](#) by Dan Mons.



In short, it had been observed that there were notable discrepancies between the RGB and YUV (composite) video outputs of the **PC Engine / TurboGrafx-16**.

My work was to check for the presence of a suspected color conversion table, and if it was, to extract its data.

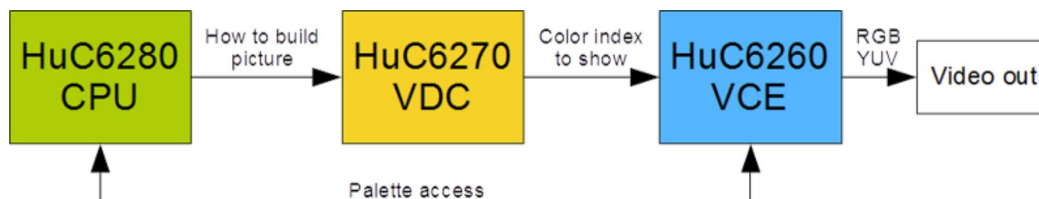
Here's a very short summary of the PC Engine's video architecture:

The CPU tells the VDC (Video Display Controller) how the picture must be built: where the sprites must be, what tiles to use, how the layers are scrolled, etc...

As the VDC produces the image, it sends a stream of pixel color indices to the VCE, which uses them as an address to look up the needed color in the palette.

The VCE (Video Color Encoder) is responsible for generating the final analog RGB and YUV video signals.

It contains a palette RAM able to store 512 9-bit RGB color definitions: 3 bits for red, 3 for green, and 3 for blue, no surprises.



The palette RAM in the VCE can be accessed by both the VDC (for rendering) and by the CPU (for setting up the colors). Priority is handled inside the VCE by a simple switch which ignores the VDC accesses when the CPU wants to read or write to the palette RAM, giving the CPU top priority.

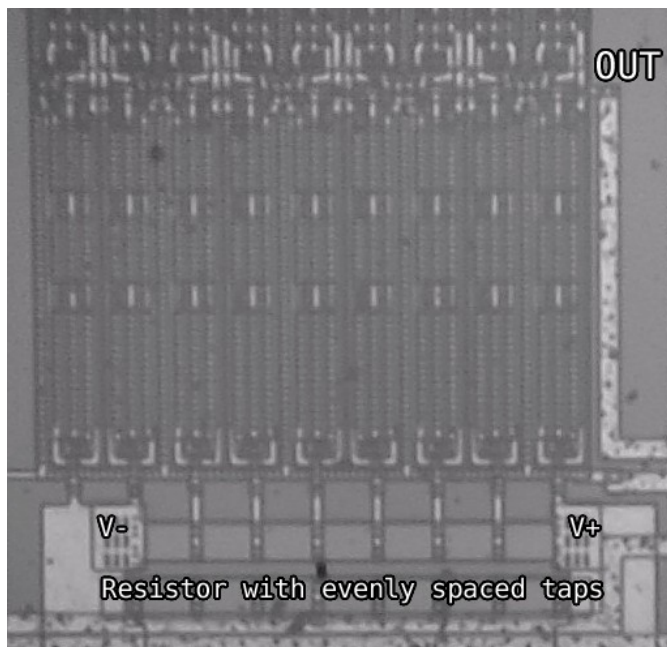
The HuC6260 of a certified dead and unfixable PC Engine console was decapped and microscope observation revealed the expected blocks: some **standard cell logic**, SRAM, DACs and less expected ROM block (!).

For the RGB output, the digital RGB data coming out of the palette RAM is directly converted to analog by three identical 3-bit DACs, one for each component.

These are string DACs (also called Kelvin dividers), where a long resistor is tapped by analog switches, one for each possible output value.

The 3-bit value is decoded to $2^3 = 8$ "one-hot" lines, each selecting an output voltage.

Here for some reason the lowest intensity value (V- on the left) uses two switches in parallel, maybe for greater current handling. In total there are 9 switches per DAC, which are the tall structures above the resistor:

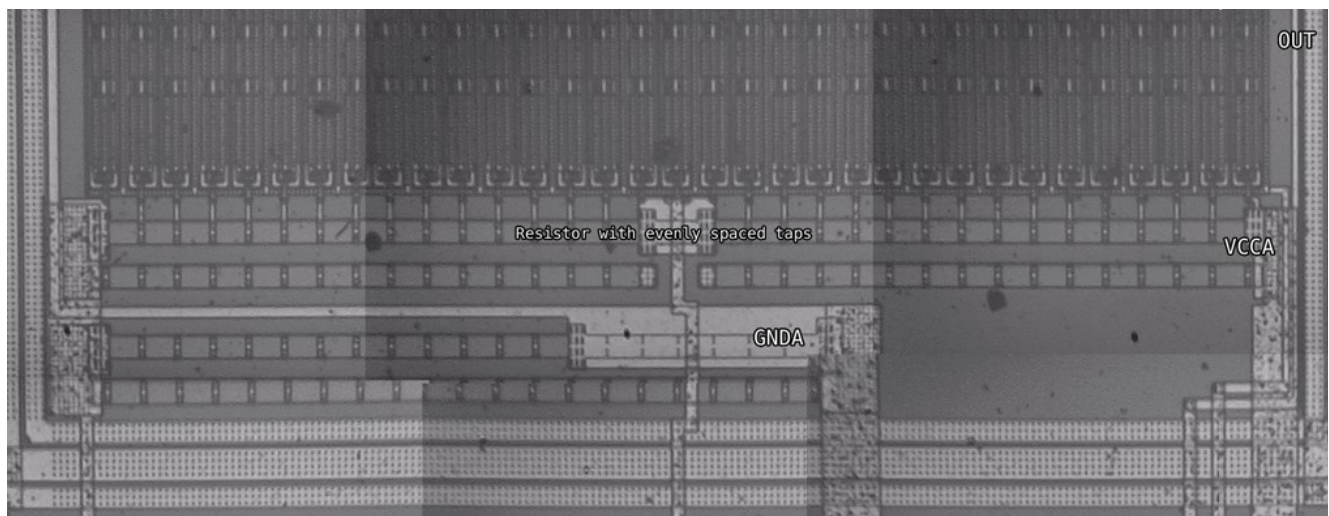


For the YUV output however, since there's no easy way to convert from RGB without using time and space consuming hardware multipliers, it was strongly suspected that a lookup table was used.

Conversion from 3-bit linear RGB components to YUV gives values that would sit in between the possible analog steps of a 3-bit DAC. Such a low resolution conversion would cause colors to be poorly mapped across the YUV gamut. In practice however, this problem was observable but not as bad. This meant that the YUV outputs were either using higher precision DACs, or non-linear ones.

Following the YUV outputs pin pads provided the answer: they were all connected to linear 5-bit DACs, which have $2^5 = 32$ steps, instead of 8 for RGB.

Here's one of the 3 YUV DACs, notice how bigger it is compared to those used for RGB:



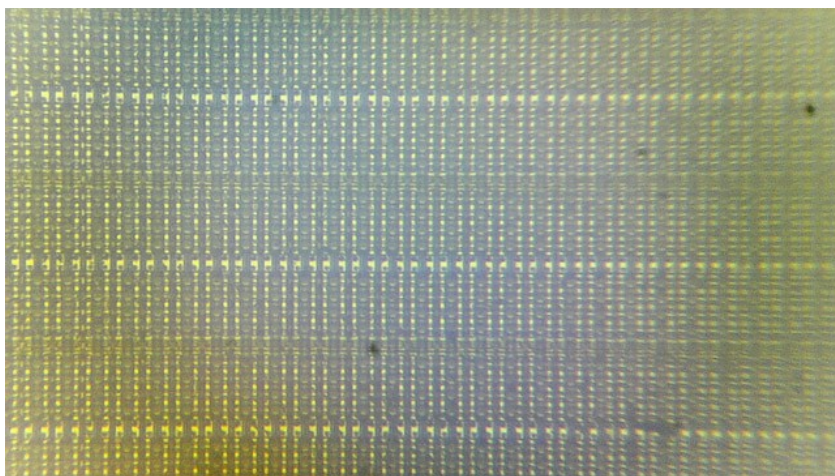
At that point, I expected the $3 * 5 = 15$ input data lines to simply come from the ROM block I had noticed earlier. Following one of those lines lead straight up to it, confirming that it held the data we were looking for.

The CPU can freely access the VCE's RGB palette memory for obvious reasons. However, being able to read or modify the YUV lookup table from the CPU would make no sense, so it was implemented as a simple ROM stuck between the palette RAM outputs and the DACs.

In such old systems, data tables are often stored as **mask ROM**, where the data bits are stored as transistors connected one way or the other.

The patterns formed by the orientation of the transistors makes optical reading of the data tedious, but possible.

Unfortunately, even by changing the light source and using the highest magnification, the ROM block present in the VCE showed no clear variations.



Obfuscation by hiding the data under one of the metal layers was very unlikely as there was no reason to, and both metal layers were obviously for used for routing anyway.

Then I remembered reading about the troubles of optically dumping **implant ROM**. This is a more advanced process used to make ROMs more compact, by storing the data as nearly invisible silicon doping differences instead of changing the shape of the layers.

The effort would have ended up failing here if there wasn't another way to read the ROM.

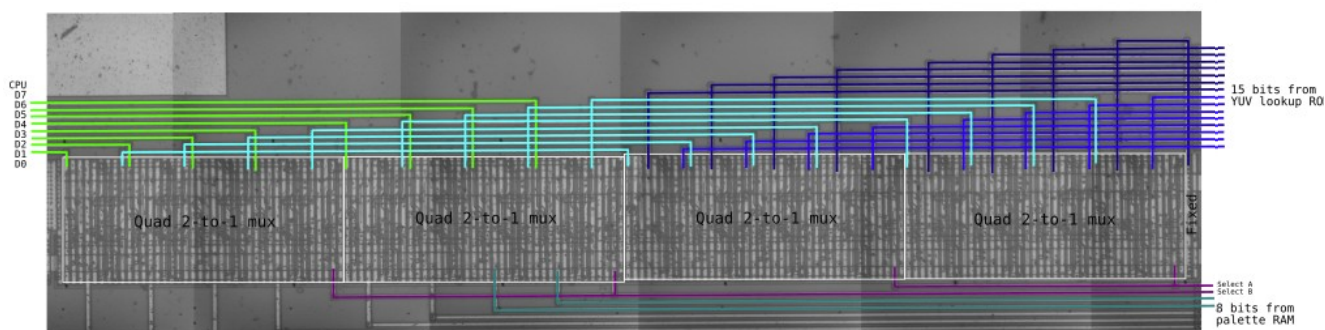
Looking back in slight despair at the data bus between the ROM and the DACs, I noticed junction points that I had first missed. The whole bus actually forked to some logic cells, meaning that some other part of the VCE was able to use the ROM's output. If that didn't lead to chopped-off test lines (some were seen in the HuC6270), this was looking a lot like a test mode that was left functional.

The ROM data lines fork to groups of 2-to-1 multiplexers forming two switching stages back to back, effectively forming 3-to-1 muxes.

The first selects the upper or the lower half of the 15 YUV ROM data bits (the 16th bit is fixed to zero), so that they fit nicely on a 8-bit bus, the same size as the CPU data bus. That's interesting.

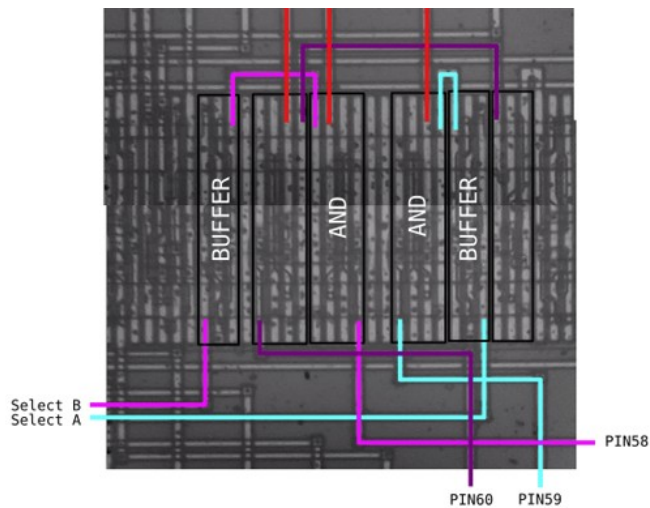
The second set of muxes select between the palette RAM data or the previously split YUV table data. Even more interesting, this means something is physically able to access both the palette data and the YUV table data.

Following the final output bus led to the CPU data pin outputs drivers, confirming that there was indeed a path between the ROM output and the CPU.



Now I needed to know how these muxes were controlled. The two purple traces labelled "Select A" and "Select B" control the two mux stages.

Following them led to a small group of logic cells, quickly identified as buffers and AND gates thanks to their small size and IO count.



The red traces are connected to a "fixed level" cell, effectively providing a constant "logic high" level.
The other inputs simply come from the pin 58 and pin 59 pads, which are coincidentally unlabelled on the PC Engine schematics **found on Console5**.

Since the AND cells have one of their inputs always set high, this circuit simply buffers the inputs from pin 58 and 59 to the Select A and Select B signals.

Pin 60 is used for another test function that didn't end up useful for our purpose.

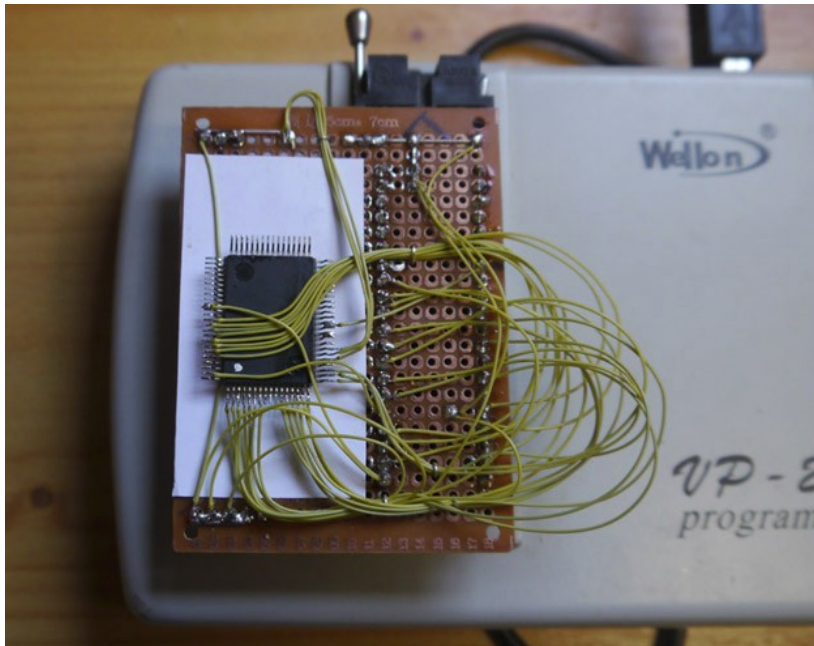
The apparently useless AND cells might have been used to provide an easy way of disabling the test pins in silicon if that needed to be done for some reason: they would have just had to set the common "high" signal to "low".

In the same way, I expected the ROM readout test mode to also route the CPU's address bus to the ROM instead of the palette RAM, to allow reading it in software.

For some unknown reason, it took me a while to understand that it was even simpler than that.

At this point I had a really bad brain fart and turned on the soldering iron instead of spending a few minutes more on the die pictures.

I undertook the task of desoldering the HuC6260 to spaghetti-wire it to a memory programmer with its test pins correctly set, in order to read it like regular ROM chip.



This didn't give any sensible values and even worse, data changed from read to read if the power was cycled. It's only a few days later that I realized what I did wrong, and that I could just have left the chip on the board to be read via the console's own CPU.

Test pins are typically used once during manufacturing and then never again. For example they may enable test modes exposing internal signals by re-purposing pins, to test long counters faster, to get characteristics of analog circuits, or... to check the integrity of internal memories.

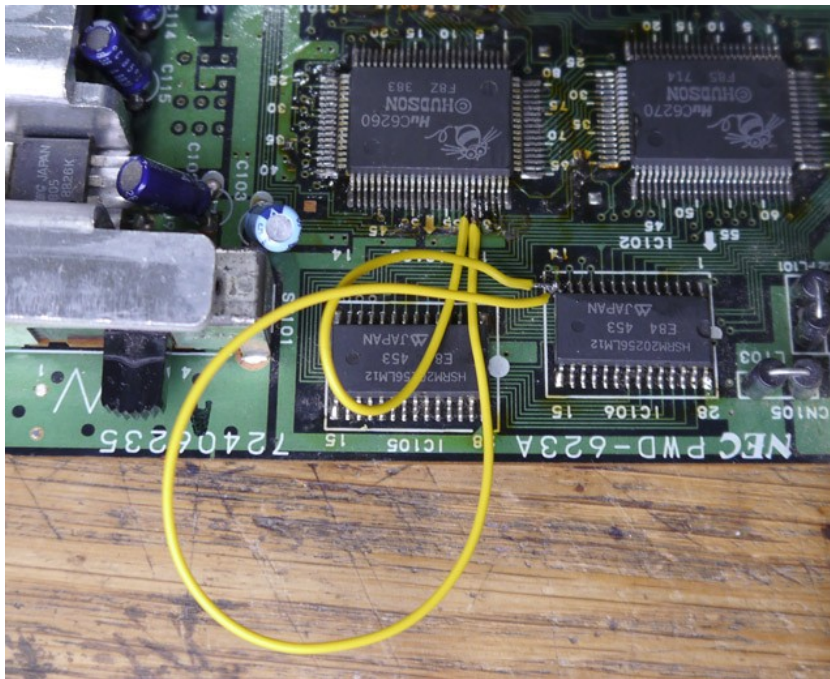
Once the chip is tested and confirmed functional, it's soldered on the board with the test pins tied to a power rail to make sure the test functions are kept disabled forever.

Because it's intended to be used only once, it's desirable for the test mode logic to take the smallest area possible to reduce

costs. Since the chip is going to be tested in a special test rig before being soldered, there's no particular need for the YUV ROM to be addressable from the CPU address bus.

So to avoid adding useless switches for the address bus on the silicon, they simply left the ROM connected to the palette RAM output.

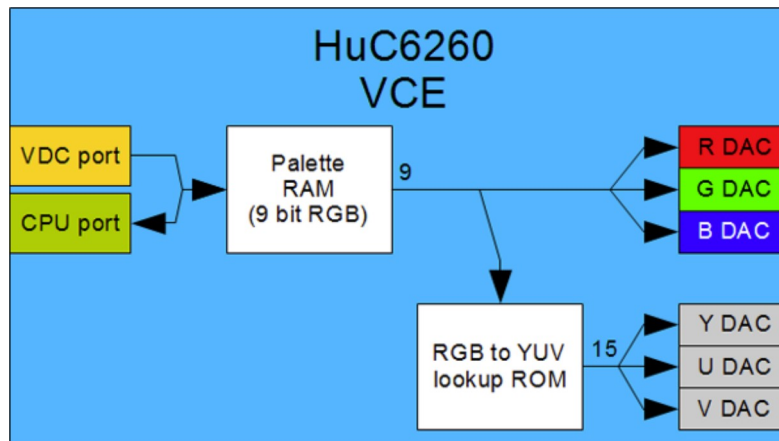
This means that instead of seeing the YUV ROM mapped in place of the palette RAM and reading it like so by the CPU as I was expecting, it is actually addressed BY the palette RAM's output, effectively doing a double lookup. It makes the reading process slightly more complicated but it allowed Hudson to save silicon area.



So with the stupid spaghetti removed and the chip soldered back on the board with the right test mode enabled, the CPU can read the YUV ROM instead of the palette RAM.

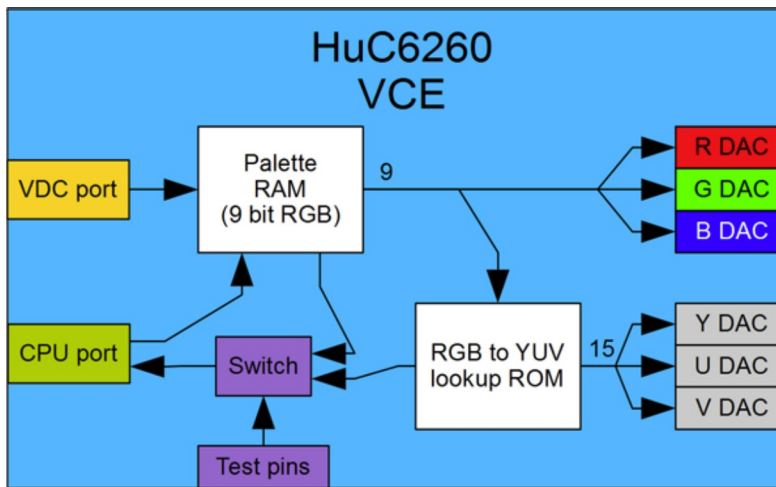
Because of the double-lookup through the palette RAM, there has to be some control over which color index is being output from the VDC when the CPU performs the read in order to read from the ROM address we want.

Here are the paths when the HuC6260 is in its normal, non-test mode:



The VDC or the CPU can set the palette RAM address, and the CPU can read from/write to it. The palette RAM outputs (the 9-bit RGB values) go to both the RGB DACs, and to the YUV ROM address inputs. The YUV ROM then outputs its data to the YUV DACs.

When the ROM dump test mode is enabled, a path appears between the YUV ROM data bus and the CPU bus. Since the ROM data is 15-bit, a second test pin is used to select between the upper or lower bits. There's no software control over this.



The simplest way I found to dump the ROM in software, is to clear the entire display so that only a single color is shown, and read the YUV ROM whenever the display is active (not in blanking periods, because the special black output could interfere).

1. Enable YUV ROM readout mode by wiring test pins as necessary.
2. Clear entire display, no sprites, no scrolling layers.
3. Wait for v-blank interrupt.
4. Set backdrop color value to the ROM address we want to read.
5. Read from the "palette RAM" which is actually the ROM data now, store value in work RAM.
6. Increment address.
7. Repeat steps 3 to 7, 512 times (epilepsy warning).
8. Setup a simple UI to visualize the 512 previously read values.

Thanks to the nice online documentation about PC-Engine programming, I was able to write a small program to perform these steps and then just slapped it on my Everdrive.

Since there's no way to control the lower/upper bits selection in software, I repeated the readback procedure with pin 59 wired to VCC instead of GND.

This gave a totally different set of values, which confirmed that the internal mux was working.

Here are the 512, 8 lower bits of the ROM. The topmost row is an hex index (\$0 to \$1F8, 24 values per column).

YUVDUMP	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0	100	110	120	130	140	150	160	170	180	190	1A0	1B0	1C0	1D0	1E0	1F0	
0	18	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0	100	110	120	130	140	150	160	170	180	190	1A0	1B0	1C0	1D0	1E0	1F0
1	19	31	41	51	61	71	81	91	A1	B1	C1	D1	E1	F1	101	111	121	131	141	151	161	171	181	191	1A1	1B1	1C1	1D1	1E1	1F1
2	1A	2A	3A	4A	5A	6A	7A	8A	9A	AA	BA	CA	DA	EA	FA	10A	11A	12A	13A	14A	15A	16A	17A	18A	19A	1AA	1BA	1CA	1DA	1FA
3	1B	2B	3B	4B	5B	6B	7B	8B	9B	AB	BB	CB	DB	EB	FB	10B	11B	12B	13B	14B	15B	16B	17B	18B	19B	1AB	1BB	1CB	1DB	1FB
4	1C	2C	3C	4C	5C	6C	7C	8C	9C	AC	BC	CC	DC	EC	FC	10C	11C	12C	13C	14C	15C	16C	17C	18C	19C	1AC	1BC	1CC	1DC	1FC
5	1D	2D	3D	4D	5D	6D	7D	8D	9D	AD	BD	CD	DD	ED	FD	10D	11D	12D	13D	14D	15D	16D	17D	18D	19D	1AD	1BD	1CD	1DD	1FD
6	1E	2E	3E	4E	5E	6E	7E	8E	9E	AE	BE	CE	DE	EE	FE	10E	11E	12E	13E	14E	15E	16E	17E	18E	19E	1AE	1BE	1CE	1DE	1FE
7	1F	2F	3F	4F	5F	6F	7F	8F	9F	AF	BF	CF	DF	EF	FF	10F	11F	12F	13F	14F	15F	16F	17F	18F	19F	1AF	1BF	1CF	1DF	1FF
8	20	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0	100	110	120	130	140	150	160	170	180	190	1A0	1B0	1C0	1D0	1E0	1F0
9	21	31	41	51	61	71	81	91	A1	B1	C1	D1	E1	F1	101	111	121	131	141	151	161	171	181	191	1A1	1B1	1C1	1D1	1E1	1F1
10	22	32	42	52	62	72	82	92	A2	B2	C2	D2	E2	F2	102	112	122	132	142	152	162	172	182	192	1A2	1B2	1C2	1D2	1E2	1F2
11	23	33	43	53	63	73	83	93	A3	B3	C3	D3	E3	F3	103	113	123	133	143	153	163	173	183	193	1A3	1B3	1C3	1D3	1E3	1F3
12	24	34	44	54	64	74	84	94	A4	B4	C4	D4	E4	F4	104	114	124	134	144	154	164	174	184	194	1A4	1B4	1C4	1D4	1E4	1F4
13	25	35	45	55	65	75	85	95	A5	B5	C5	D5	E5	F5	105	115	125	135	145	155	165	175	185	195	1A5	1B5	1C5	1D5	1E5	1F5
14	26	36	46	56	66	76	86	96	A6	B6	C6	D6	E6	F6	106	116	126	136	146	156	166	176	186	196	1A6	1B6	1C6	1D6	1E6	1F6
15	27	37	47	57	67	77	87	97	A7	B7	C7	D7	E7	F7	107	117	127	137	147	157	167	177	187	197	1A7	1B7	1C7	1D7	1E7	1F7
16	28	38	48	58	68	78	88	98	A8	B8	C8	D8	E8	F8	108	118	128	138	148	158	168	178	188	198	1A8	1B8	1C8	1D8	1E8	1F8
17	29	39	49	59	69	79	89	99	A9	B9	C9	D9	E9	F9	109	119	129	139	149	159	169	179	189	199	1A9	1B9	1C9	1D9	1E9	1F9
18	2A	3A	4A	5A	6A	7A	8A	9A	AA	BA	CA	DA	EA	FA	10A	11A	12A	13A	14A	15A	16A	17A	18A	19A	1AA	1BA	1CA	1DA	1FA	100
19	2B	3B	4B	5B	6B	7B	8B	9B	AB	BB	CB	DB	EB	FB	10B	11B	12B	13B	14B	15B	16B	17B	18B	19B	1AB	1BB	1CB	1DB	1FB	101
20	2C	3C	4C	5C	6C	7C	8C	9C	AC	BC	CC	DC	EC	FC	10C	11C	12C	13C	14C	15C	16C	17C	18C	19C	1AC	1BC	1CC	1DC	1FC	102
21	2D	3D	4D	5D	6D	7D	8D	9D	AD	BD	CD	DD	ED	FD	10D	11D	12D	13D	14D	15D	16D	17D	18D	19D	1AD	1BD	1CD	1DD	1FD	103
22	2E	3E	4E	5E	6E	7E	8E	9E	AE	BE	CE	DE	EE	FE	10E	11E	12E	13E	14E	15E	16E	17E	18E	19E	1AE	1BE	1CE	1DE	1FE	104
23	2F	3F	4F	5F	6F	7F	8F	9F	AF	BF	CF	DF	EF	FF	10F	11F	12F	13F	14F	15F	16F	17F	18F	19F	1AF	1BF	1CF	1DF	1FF	105
24	30	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0	100	110	120	130	140	150	160	170	180	190	1A0	1B0	1C0	1D0	1E0	1F0	106
25	31	41	51	61	71	81	91	A1	B1	C1	D1	E1	F1	101	111	121	131	141	151	161	171	181	191	1A1	1B1	1C1	1D1	1E1	1F1	107
26	32	42	52	62	72	82	92	A2	B2	C2	D2	E2	F2	102	112	122	132	142	152	162	172	182	192	1A2	1B2	1C2	1D2	1E2	1F2	108
27	33	43	53	63	73	83	93	A3	B3	C3	D3	E3	F3	103	113	123	133	143	153	163	173	183	193	1A3	1B3	1C3	1D3	1E3	1F3	109
28	34	44	54	64	74	84	94	A4	B4	C4	D4	E4	F4	104	114	124	134	144	154	164	174	184	194	1A4	1B4	1C4	1D4	1E4	1F4	110
29	35	45	55	65	75	85	95	A5	B5	C5	D5	E5	F5	105	115	125	135	145	155	165	175	185	195	1A5	1B5	1C5	1D5	1E5	1F5	111
30	36	46	56	66	76	86	96	A6	B6	C6	D6	E6	F6	106	116	126	136	146	156	166	176	186	196	1A6	1B6	1C6	1D6	1E6	1F6	112
31	37	47	57	67	77	87	97	A7	B7	C7	D7	E7	F7	107	117	127	137	147	157	167	177	187	197	1A7	1B7	1C7	1D7	1E7	1F7	113
32	38	48	58	68	78	88	98	A8	B8	C8	D8	E8	F8	108	118	128	138	148	158	168	178	188	198	1A8	1B8	1C8	1D8	1E8	1F8	114
33	39	49	59	69	79	89	99	A9	B9	C9	D9	E9	F9	109	119	129	139	149	159	169	179	189	199	1A9	1B9	1C9	1D9	1E9	1F9	115
34	3A	4A	5A	6A	7A	8A	9A	AA	BA	CA	DA	EA	FA	10A	11A	12A	13A	14A	15A	16A	17A	18A	19A	1AA	1BA	1CA	1DA	1FA	100	116
35	3B	4B	5B	6B	7B	8B	9B	AB	BB	CB	DB	EB	FB	10B	11B	12B	13B	14B	15B	16B	17B	18B	19B	1AB	1BB	1CB	1DB	1FB	101	117
36	3C	4C	5C	6C	7C	8C	9C	AC	BC	CC	DC	EC	FC	10C	11C	12C	13C	14C	15C	16C	17C	18C	19C	1AC	1BC	1CC	1DC	1FC	102	118
37	3D	4D	5D	6D	7D	8D	9D	AD	BD	CD	DD	ED	FD	10D	11D	12D	13D	14D	15D	16D	17D	18D	19D	1AD	1BD	1CD	1DD	1FD	103	119
38	3E	4E	5E	6E	7E	8E	9E	AE	BE	CE	DE	EE	FE	10E	11E	12E	13E	14E	15E	16E	17E	18E	19E	1AE	1BE	1CE	1DE	1FE	104	120
39	3F	4F	5F	6F	7F	8F	9F	AF	BF	CF	DF	EF	FF	10F	11F	12F	13F	14F	15F	16F	17F	18F	19F	1AF	1BF	1CF	1DF	1FF	105	121
40	40	50	60	70	80	90	A0	B0	C0	D0	E0	F0	100	110	120	130	140	150	160	170	180	190	1A0	1B0	1C0	1D0	1E0	1F0	106	122
41	41	51	61	71	81	91	A1	B1	C1	D1	E1	F1	101	111	121	131	141	151	161	171	181	191	1A1	1B1	1C1	1D1	1E1	1F1	107	123
42	42	52	62	72	82	92	A2	B2	C2	D2	E2	F2	102	112	122	132	142	152	162	172	182	192	1A2	1B2	1C2	1D2	1E2	1F2	108	124
43	43	53	63	73	83	93	A3	B3	C3	D3	E3	F3	103	113	123	133	143	153	163	173	183	193	1A3	1B3	1C3	1D3	1E3	1F3	109	125
44	44	54	64	74	84	94	A4	B4	C4	D4	E4	F4	104	114	124	134	144	154	164	174	184	194	1A4	1B4	1C4	1D4	1E4	1F4	110	126
45	45	55	65	75	85	95	A5	B5	C5	D5	E5	F5	105	115	125	135	145	155	165	175	185	195	1A5	1B5	1C5	1D5	1E5	1F5	111	127
46	46	56	66	76	86	96	A6	B6	C6	D6	E6	F6	106	116	126	136	146	156	166	176	186	196	1A6	1B6	1C6	1D6	1E6	1F6	112	128
47	47	57	67	77	87	97	A7	B7	C7	D7	E7	F7	107	117	127	137	147	157	167	177	187	197	1A7	1B7	1C7	1D7	1E7	1F7	113	129
48	48	58	68	78	88	98	A8	B8	C8	D8	E8	F8	108	118	128	138	148	158	168	178	188	198	1A8	1B8	1C8	1D8	1E8	1F8	114	130

